TUTORIAL – 12

AI-Powered Software Testing

Aim:

To analyze AI-powered software testing tools and apply Generative AI techniques for test case generation, test data creation, defect prediction, requirement analysis and automated Selenium test script generation for software quality assurance.

Description:

Generative AI can assist software testers in analyzing requirements and generating test scenarios, test cases, test data and automation scripts.

In this practical, students will use a Generative AI tool as an **assistant** and then execute and verify the generated Selenium Python code in **PyCharm**.

AI output must be reviewed and corrected before execution.

Problem Statement:

Consider the following requirement:

> A registered user should be able to log in using valid username and password. Invalid credentials should be rejected and an appropriate error message should be displayed.

Use Generative AI to:

1. Analyze the requirement.
2. Generate positive test cases.
3. Generate negative test cases.
4. Generate test data.
5. Suggest possible defect-prone areas.
6. Generate Selenium Python code.
7. Review the generated code.
8. Execute the final code in PyCharm.
9. Prepare regression test scenarios.

Implementation Details

Parameter	Details
Project Name	AI-Powered Software Testing
Application/Software	SauceDemo
Module Name	Login and Authentication
SDLC Model	Agile

Programming Language	Python
IDE/Tool Used	PyCharm
AI Tool	Generative AI
Automation Tool	Selenium WebDriver
Input/Test Data	Login Requirement and Test Credentials

Automation Test Configuration

Parameter	Details
Automation Tool	Selenium WebDriver
Python Version	Python 3.x
Selenium Version	Installed using pip
WebDriver Used	Selenium-managed Chrome WebDriver
Browser	Google Chrome
Target Website URL	https://www.saucedemo.com/
Test Framework	Selenium + Python
Execution Date	_____

Procedure – Generative AI Test Case Generation

Step 1

Open a Generative AI tool.

Step 2

Enter the requirement.

Step 3

Use the following prompt:

Analyze this software requirement:

A registered user should be able to log in using a valid username and password. Invalid credentials should be rejected and an appropriate error message should be displayed.

Generate:
1. Positive test cases
2. Negative test cases
3. Test data
4. Expected results
5. Regression test scenarios
6. Possible defect-prone areas
7. Selenium Python automation code

The final Selenium code should be executable in PyCharm.

Step 4
Review the AI output.
Step 5
Check:

- Test cases
- Test data
- Expected results
- Selenium locators
- Python syntax
- Assertions
- Error handling

Step 6
Copy the **reviewed and corrected** Selenium code into PyCharm.
Step 7
Execute the Python file.

AI-Generated Test Case Details

Test Case ID	Scenario	Test Data	Expected Result
TC-12-01	Valid Login	standard_user / secret_sauce	Login successful
TC-12-02	Invalid Password	standard_user / wrong123	Error displayed
TC-12-03	Invalid Username	wrong_user / secret_sauce	Error displayed
TC-12-04	Empty Username	Blank / secret_sauce	Validation message
TC-12-05	Empty Password	standard_user / Blank	Validation message

AI-Based Requirement Analysis

Requirement	Testable Scenario
Valid user can log in	Valid Login
Invalid credentials rejected	Negative Login
Error should be displayed	Error Message Validation
Login must use username/password	Form Validation
Existing login must continue working	Regression Testing

AI-Based Defect Prediction

Students should ask Generative AI:
Analyze a login module and identify possible defect-prone areas based on input validation, authentication, error handling, session management and user interaction.
Give reasons for each prediction.

Students must record the AI-generated suggestions and verify whether they are technically reasonable.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.maximize_window()

try:

    driver.get("https://www.saucedemo.com/")

    wait = WebDriverWait(driver, 10)

    username = wait.until(
        EC.visibility_of_element_located(
            (By.ID, "user-name")
```

```
        )
    )

    password = driver.find_element(
        By.ID, "password"
    )

    login_button = driver.find_element(
        By.ID, "login-button"
    )

    username.send_keys("standard_user")
    password.send_keys("secret_sauce")
    login_button.click()

    wait.until(
        EC.url_contains("inventory")
    )

    assert "inventory" in driver.current_url

    print("AI-Assisted Selenium Test: PASS")
    print("Requirement Validation: PASS")

except AssertionError:
    print("AI-Assisted Selenium Test: FAIL")

finally:
    driver.quit()
```

Automation Test Case Details

Field	Details
Test Case ID	TC-12-01
Module Name	Login
Automation Script Name	ai_login_test.py
Test Scenario	Valid Login

Test Objective	Verify valid login
Test Data	standard_user / secret_sauce
Expected Result	User reaches Products page
Actual Result	Products page displayed
Status	Pass
Executed By	_____
Execution Date	_____

Automation Test Execution

Step No.	Automation Action	Expected Result	Actual Result	Status
1	Open website	Login page displayed	Displayed	Pass
2	Enter username	Username accepted	Accepted	Pass
3	Enter password	Password accepted	Accepted	Pass
4	Click login	Products page opens	Opened	Pass
5	Validate URL	Inventory URL displayed	Displayed	Pass

Regression Test Execution Report

Regression Test ID	Module	Scenario	Expected Result	Actual Result	Status
RT-01	Login	Valid Login	Successful Login	Successful	Pass
RT-02	Login	Invalid Password	Error displayed	Error displayed	Pass
RT-03	Login	Invalid Username	Error displayed	Error displayed	Pass
RT-04	Login	Empty Username	Validation displayed	Validation displayed	Pass

Automation Execution Summary

Metric	Value
Total Automation Scripts	1
Scripts Executed	1
Passed	1
Failed	0
Regression Tests Executed	4
Execution Time	_____
Overall Status	PASS

Observation

Sr. No.	Observation	Remarks
1	Requirement analyzed using Generative AI	Completed
2	Test cases generated	Completed
3	Test data generated	Completed
4	Selenium script generated/reviewed	Completed
5	Selenium script executed in PyCharm	Successful

Output

Students should capture:

1. AI prompt
2. AI-generated test cases
3. AI-generated test data
4. AI-generated/reviewed Selenium code
5. PyCharm source code
6. Browser execution
7. PyCharm console output

Questionnaire

Q1. Which technology is primarily used in this practical to automate web browsers?

Answer: C) Selenium WebDriver

Q2. What is the main advantage of using Generative AI for test case generation?

Answer: C) It speeds up creation of test cases from requirements.

Q3. Which testing artifact can Generative AI create from a login requirement?

Answer: C) Selenium automation script

Q4. Defect prediction is mainly used to:

Answer: B) Identify modules likely to contain defects

Q5. Why should AI-generated test cases and scripts still be reviewed by humans?

Answer: C) Generated artifacts may contain errors, gaps, or incorrect assumptions.